

Rapport de Projet BDD + Python : Com'Art

Gestion de la production et de la distribution de contenu

Licence MIASHS L-2

Membre du groupe :

ALTANTUYA Tsogt-Erdene

HAMIDI Mariam

KUDUZOVIC Zerine

PERRIN Clément

Sommaire :

1. Présentation du projet
2. Modélisation
 - a. Modèle conceptuelle/Modèle physique
 - b. Domain(classes métiers)/DAOs
3. Descriptions des fonctionnalités
 - a. Fonctionnement des modules clés
 - b. Localisation des requêtes SQL
4. Tests et validation
 - a. Approche de test
 - b. Résultats
 - c. Améliorations possibles
5. Usage de l'IA
6. Difficultés rencontrées et solutions

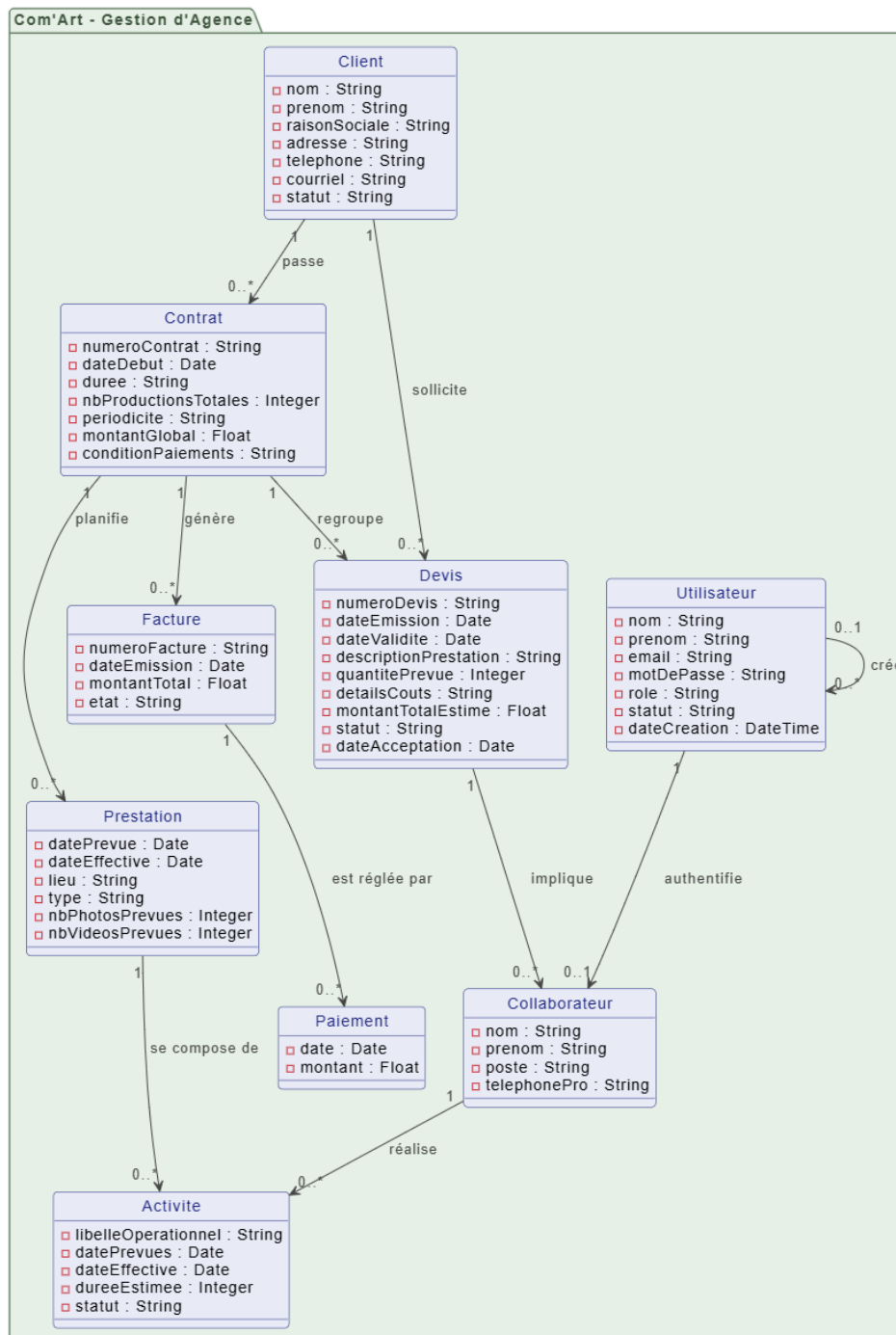
1. Introduction

Notre projet Com'Art vise à développer une application de gestion pour une agence de communication spécialisée dans la création de contenus visuels comme des photos et des vidéos pour des clients particuliers et professionnels.

L'objectif principal du projet est de développer une application complète en centralisant toutes les activités de l'agence dans un seul système informatique afin de faciliter le suivi commercial, l'organisation des prestations, la gestion des contrats, la facturation ainsi que le suivi des paiements.

2. Modélisation de la Base de Données et Architecture

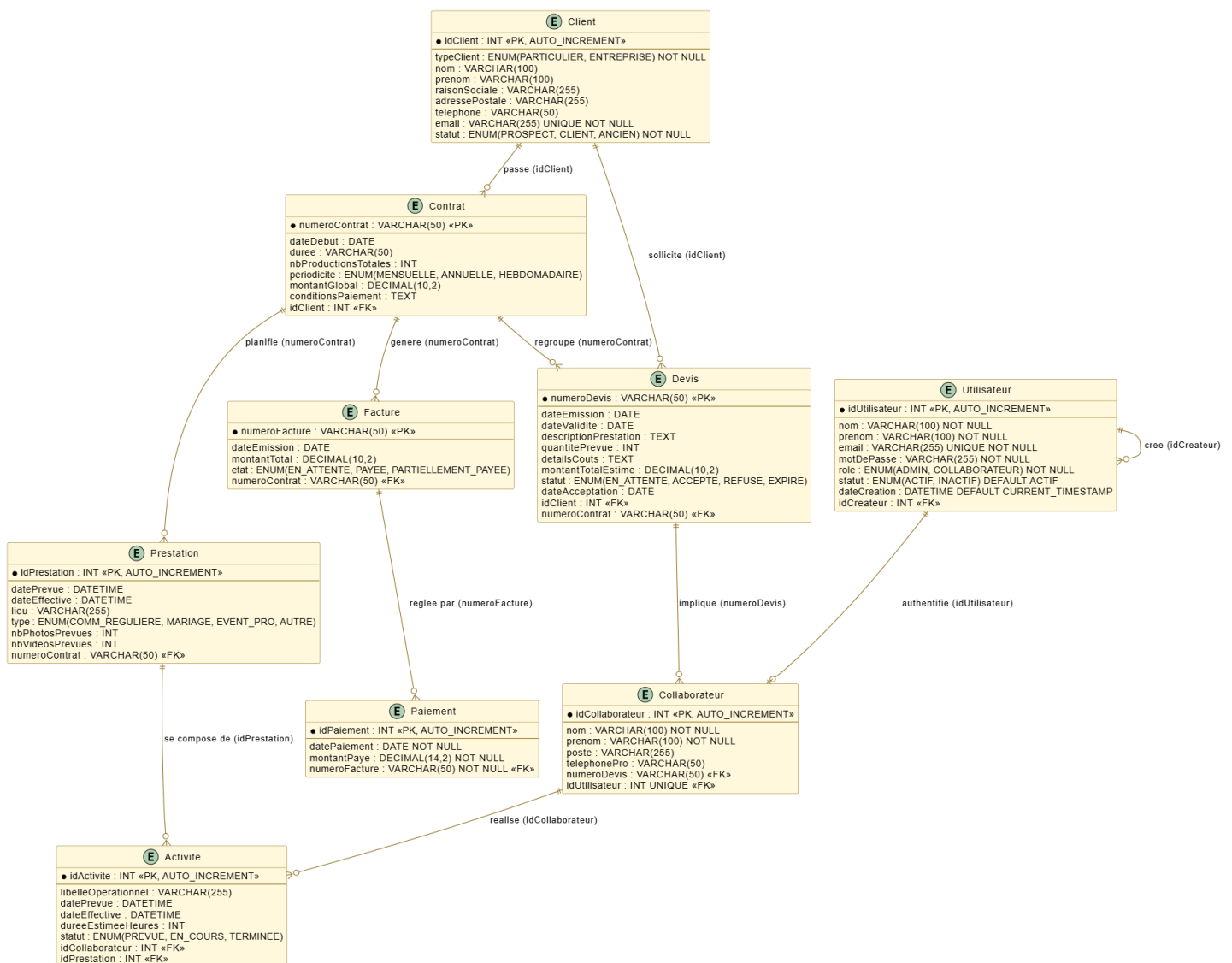
2.1 Modèle Conceptuel de Données (MCD)



Notre modèle conceptuel est basé sur les règles de gestion définies dans l'énoncé :

- Un **Client** peut solliciter plusieurs demandes de **Devis**.
- Un **Devis**, une fois accepté, donne lieu à la création d'un unique **Contrat** (relation 1:1).
- Le **Contrat** est l'élément central : il regroupe une ou plusieurs **Prestations** et génère une ou plusieurs **Factures**.
- Une **Prestation** est composée de plusieurs **Activités** opérationnelles. Chaque activité est réalisée par un **Collaborateur**.
- Une **Facture** est soldée par un ou plusieurs **Paiements**.

2.2 Modèle Physique de Données (MPD)



Nous avons préféré utiliser des énumérations (ENUM) pour les statuts (ex: statut d'un devis : EN_ATTENTE, ACCEPTE, REFUSE, EXPIRE) pour verrouiller les données insérées directement au niveau de la base de données.

2.3 Architecture Backend : Data Access Objects (DAO)

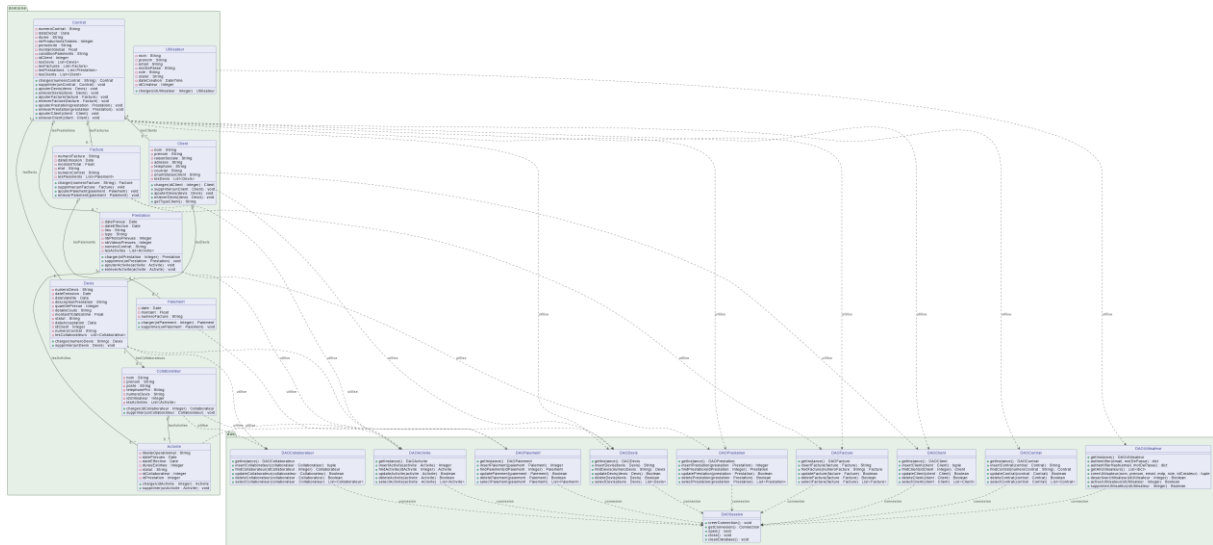
Pour séparer notre interface graphique de la base de données, nous avons implémenté le pattern **DAO**.

- **DAOSession** : Gère la connexion unique à MySQL (mysql-connector-python) et centralise les transactions (commit/rollback). Une méthode `clean_database()` y est implémentée et est très utile pour nos phases de tests.
- **Sécurité et requêtes dynamiques** : Chaque DAO implémente un CRUD complet. Pour prévenir les injections SQL, toutes les requêtes sont paramétrées. Les méthodes de recherche (select) construisent dynamiquement la clause WHERE en fonction des attributs de l'objet fourni.
- **Bonus Sécurité (DAOUtilisateur)** : Pour la gestion des accès à l'application, les mots de passe des collaborateurs ne sont pas stockés en clair. Nous avons utilisé la bibliothèque hashlib (SHA-256) lors de l'insertion et de l'authentification (`authentifier_hash`).

2.4 Modèle de programmation : Classes Métiers (Domaine)

Le dossier domaine contient nos entités métiers (Client, Contrat, ...), elles permettent :

- **Encapsulation et Validation** : Tous les attributs sont privés. Les *setters* imposent des validations de type (isinstance) et de logique métier (ex: interdiction d'avoir un nombre de photos négatif dans une Prestation).
- **Synchronisation (Active-Record partiel)** : L'instanciation d'un nouvel objet sans ID déclenche automatiquement son insertion en base via son DAO respectif. Chaque modification par un *setter* met à jour la base de données en temps réel.
- **Chargement en cascade** : Les méthodes statiques `charger(id)` permettent de reconstruire l'arbre complet d'un objet (ex: charger un contrat récupère automatiquement ses prestations et factures associées).



3. Description des fonctionnalités Frontend (Tkinter)

3.1 Navigation et Ergonomie

L'application repose sur une fenêtre unique (MainWindow) qui remplace son contenu dynamiquement au lieu d'ouvrir plusieurs fenêtres. L'affichage s'adapte au rôle de la personne connectée (seul le rôle ADMIN voit les boutons de suppression et la gestion des utilisateurs).

L'affichage des données se fait via des composants ttk.Treeview avec deux barres de défilement, et les ajouts/modifications se font via des formulaires modaux (tk.Toplevel) bloquant le reste de l'interface (grab_set) pour garantir la finalisation de l'action.

3.2 Implémentation du Workflow Métier

Nous avons automatisé les règles de gestion compliquées directement dans l'interface :

- **Génération dynamique** : Les formulaires s'adaptent (ex: cacher les champs "Raison Sociale" si le type de client sélectionné est "PARTICULIER").
- **Devis vers Contrat** : Lorsqu'un devis passe au statut "ACCEPTE", l'interface ouvre automatiquement une modale obligeant l'utilisateur à lier ce devis à un contrat existant ou à en créer un nouveau, ce qui génère automatiquement un numéro de contrat formaté (CONT-2026-XXX).
- **Génération automatique des Factures** : La méthode `generer_factures_manquantes` vérifie directement grâce à une expression régulière (`re.search`) les conditions de paiement d'un contrat (ex: "Paiement en 3 fois") pour générer et diviser automatiquement les factures manquantes. Le "Reste à payer" est calculé dynamiquement en soustrayant la somme des paiements déjà effectués au montant de la facture.

3.3 Import et Export CSV (Module Pandas)

L'application permet l'import et l'export complet de la base de données en CSV.

- **Exports complexes** : En plus des exports simples par table, la fonction `exportation_combined_csv` utilise la bibliothèque pandas. Elle fait des jointures (merge)

entre les différents fichiers pour créer des vues plus spécifiques (ex: un fichier `factures_full.csv` contenant directement le nom du client lié).

4. Tests et Validation

Pour les tests, nous avons vérifié le bon déroulement du backend vis-à-vis des différentes erreurs possibles :

- **Tests de contraintes d'intégrité** : Tentative de suppression d'un client possédant déjà un devis. Notre code lève correctement une exception métier ("`Erreur_suppression_client_avec_devis`") et le DAO effectue un `rollback()` si l'erreur vient de la BDD.
- **Tests de saisie Tkinter** : Soumission de formulaires vides, de chaînes de caractères au lieu de nombres (ex: pour le montant d'un paiement), ou saisie d'un paiement supérieur au "reste à payer" de la facture. L'interface identifie ces erreurs et affiche des boîtes de dialogue (`messagebox.showwarning`) sans faire planter l'application.
- **Tests d'importation** : Le script d'importation globale a été testé avec une base vide (`clean_database()`) pour vérifier que les clés étrangères se reconnectaient correctement après le chargement des fichiers CSV.

5. Usage des outils IA

L'IA a été utilisée quand nous avons eu de gros blocages pour ne pas rester bloqués.

Plus précisément, l'IA a été utilisée dans les cas suivants :

- **Compréhension des patrons de conception (Design Patterns)** : L'architecture de l'application graphique couplée à la base de données a été assez compliquée à comprendre. Nous avons utilisé l'IA pour nous aider à assimiler et mettre en place une structure s'approchant du motif **MVC (Modèle-Vue-Contrôleur)**.
- **Maîtrise des Callbacks sous Tkinter** : L'IA nous a aussi aidés à comprendre la logique des *callbacks* (fonctions passées en paramètres). C'est grâce à ces explications que nous avons pu implémenter une navigation fluide et dynamique entre nos différentes fenêtres (via les commandes `on_back` et les `lambda`), sans avoir à instancier de multiples fenêtres simultanément.
- **Aide à la résolution de bugs (Déblocage)** : Lors de "grosses impasses" techniques (erreurs de syntaxe difficiles à tracer, gestion des rafraîchissements de tableaux Treeview après la fermeture d'une popup, ou problèmes d'importations entre nos classes), l'IA nous a aidé à trouver l'origine du problème et des solutions à ces derniers.

6. Difficultés rencontrées et Solutions

1. **Les imports circulaires en Python** :

- Problème : Nos classes du domaine devaient s'instancier entre elles (ex: Client chargeant une liste de Devis, et Devis ayant besoin de connaître Client), ce qui générerait des erreurs `ImportError: cannot import name`.
- Solution : Nous avons déplacé certains imports directement à l'intérieur des méthodes spécifiques (comme `set_all_values` ou `charger`) plutôt qu'en haut des fichiers, brisant ainsi la boucle d'importation circulaire au chargement du module.

2. Gestion de l'état asynchrone dans l'interface graphique :

- Problème : Synchronisation de l'affichage après une modification en base de données. Initialement, lors de l'ajout d'une prestation ou d'un devis via une fenêtre `Toplevel`, le `Treeview` de la fenêtre parente restait figé sur les anciennes données.
- Solution : Nous avons implémenté un système de fonctions de rappel (callbacks). Lors de l'ouverture d'une fenêtre de saisie, nous lui transmettons en paramètre la méthode de rafraîchissement de la vue parente. Une fois l'insertion confirmée en base de données, la fenêtre secondaire exécute ce callback avant de se fermer. Pour sécuriser ce processus, nous avons utilisé `grab_set()` : cette méthode rend la fenêtre de saisie "modale", empêchant toute interaction incohérente avec le menu principal tant que l'opération n'est pas validée ou annulée.

3. Calcul du reste à payer en temps réel :

- Problème : Stocker le "reste à payer" en base de données risquait de créer des désynchronisations.
- Solution : Nous avons choisi de le calculer à la volée dans l'interface (`Facture.get_montant_total() - somme_des_paiements`), garantissant ainsi une information toujours exacte, qu'un paiement soit ajouté, modifié ou supprimé.